# Ders 5 Çalışma Özeti

## Ders 5 Çalışma Özeti

### Genel Konular

- Önceki dersteki asallık testi akış diyagramının düzeltilmesi.
- Özyineleme (recursion) tekrarı: durma koşulu (base condition), kendini çağırma (self call), çağrı yığını (call stack) ve stack overflow.
- Özyinelemeli örnekler: iki sayının toplama ile çarpılması, rakamlar toplamı, palindrom kontrolü (sayıyı ters çevirme), asallık testi.
- Alıştırma: bir dizinin $r$'li kombinasyonlarını özyinelemeli yazdırma.
- Bağlı liste (linked list): diziyle karşılaştırma, düğüm (node) yapısı, `head` ve `tail`.
- Bağlı liste türleri: tek yönlü (singly), çift yönlü (doubly), dairesel (circular) ve dairesel çift yönlü.
- Bağlı liste işlemleri: gezinme (traverse), arama (search), güncelleme (update), ekleme (insert), silme (delete).
- Bağlı listelerin gerçek hayattaki kullanım alanları.

### Hocanın Özellikle Vurguladığı Kısımlar

- Bu örneklerin amacı özyinelemenin verimliliğini göstermek değil. Amaç bir problemi özyinelemeli olarak nasıl tasarlayacağınızı göstermek.
- Her özyinelemeli fonksiyonda bir durma koşulu ve bir kendini çağırma adımı olmalı.
- Kendini çağırırken parametre mutlaka değiştirilmeli (azaltılmalı, 10'a bölünmeli vb.). Parametreyi aynen geçirmek durma koşuluna ulaşmayı sağlamaz.
- Özyineleme ne kadar derine inerse yığında o kadar yer kullanılır. Yığın taşarsa stack overflow ve segmentation fault oluşur. Yığın boyutunuzu bilin.
- Çağrı yığını LIFO (son giren ilk çıkar) çalışır. Değerler en alttaki durma koşulundan yukarı doğru döner.
- Bağlı listede son düğümün `next`'i mutlaka `NULL` olmalı. Yoksa liste hiç bitmez.
- `head` her zaman ilk düğümün adresini tutar. Bu adres kaybedilirse listeye erişilemez.
- C'de silinen düğümün belleği mutlaka `free` ile serbest bırakılmalı. Java'da bunu çöp toplayıcı (garbage collector) yapar, C'de programcının sorumluluğundadır.
- İlk düğümü silerken adresi önce geçici bir değişkene (`tmp`) alınmalı. Yoksa `head` güncellendiği anda o düğümün adresi kaybolur ve `free` yapılamaz.
- Bağlı liste sıralı da sırasız da olabilir. Sıralı listede aranan değer geçildiyse aramayı sürdürmeye gerek yoktur.
- Küçük verilerde bağlı listeyi tercih etmek için bir neden yok. Bağlı liste büyük verilerde ve çok ekleme/silme yapılan işlerde öne çıkar.
- Çift yönlü bağlı listede ekleme, silme, güncelleme ve aramayı kendiniz yazmanız önerildi.

### Kısa Tekrar Notları

- Çarpma: `product(x, y) = x + product(x, y - 1)`, durma koşulu `y == 0` için 0. Küçük sayı kadar adım atmak için gerekirse parametrelerin yeri değiştirilir.
- Rakamlar toplamı: `n % 10 + sumOfDigits(n / 10)`, durma koşulu `n == 0` için 0. 1234 için sonuç 10.
- Palindrom: sayı `static` bir toplamla ters çevrilir, ana programda orijinal sayıyla karşılaştırılır. 2002 palindromdur, 1234 değildir.

- Özyinelemeli asallık: `isPrime(n, i)`, ilk çağrı `i = 2` ile yapılır. `n % i == 0` ise asal değil, `i * i > n` ise asal, yoksa `isPrime(n, i + 1)`.
- Dizi: bellekte kesintisiz (contiguous) yer, doğrudan erişim (direct access), ama ekleme/silmede kaydırma gerekir.
- Bağlı liste: düğümler bellekte dağınık durabilir, her düğüm değer ve sonraki düğümün adresini tutar. Ek adres bilgisi bir maliyettir.
- Arama ve güncelleme dizide daha verimlidir. Bağlı listede bütün düğümler gezilmek zorunda kalınabilir.
- Ekleme ve silme bağlı listede kaydırma gerektirmez, yalnız bağlantılar değiştirilir.
- Eklemenin üç durumu: başa, araya, sona. Silmenin durumları: liste boş, ilk düğüm, ara düğüm, son düğüm, değer listede yok.
- Araya ekleme ve silmede iki işaretçi gerekir: `prev` (önceki) ve `current` (o anki).
- Dairesel listede gezinme `NULL`'a göre değil, başa (`head`) dönülüp dönülmediğine göre durdurulur.

## Detaylı Açıklamalar

### Asallık testindeki düzeltme

- Bir öğrenci önceki derste gösterilen akış diyagramındaki hatayı bildirdi.
- Akış diyagramının sonundaki "`i` eşit mi `x`" kontrolü yanlıştı. Bu kontrol, sayının kendisine kadar giden eski sürümden kalmıştı.
- Döngü $\sqrt{x}$'e kadar gittiği için sonuç `x % i` ile kontrol edilmeli. Sıfırdan farklıysa sayı asaldır.
- Döngü koşulundaki küçük-eşit de küçüktür olarak düzeltildi.

### Özyineleme tekrarı

- Önceki derste faktöriyel örneği işlenmişti. `n <= 1` için 1 döner, yoksa `n * fact(n - 1)` hesaplanır.
- Özyinelemeli fonksiyon, dönülecek noktayı yığında saklar. Her çağrı yığında yer kaplar.
- Kullanım alanları: ağaç ve graf algoritmaları, geri izleme (backtracking).

### Örnek 1: İki sayıyı toplama ile çarpmak

- Çarpma, bir sayının diğeri kadar kendisiyle toplanmasıdır. $5 \times 3 = 5 + 5 + 5 = 15$.
- 50'yi 5 kez toplamak, 5'i 50 kez toplamaktan akıllıcadır. Bu yüzden küçük sayı ikinci parametreye alınır.

```
int product(int x, int y)
{
    if (x < y)
        return product(y, x);
    if (y != 0)
        return x + product(x, y - 1);
    return 0;
}
```

- `product(50, 5)`: $50 + product(50, 4)$, $50 + 50 + product(50, 3)$, ... sonunda `y == 0` için 0 döner. Toplam 250.

### Örnek 2: Rakamlar toplamı

```
int sumOfDigits(int n)
{
    if (n == 0)
        return 0;
    return n % 10 + sumOfDigits(n / 10);
}
```

| Çağrı | `n % 10` | Sonraki çağrı |
|---|---|---|
| `sumOfDigits(1234)` | 4 | `sumOfDigits(123)` |
| `sumOfDigits(123)` | 3 | `sumOfDigits(12)` |
| `sumOfDigits(12)` | 2 | `sumOfDigits(1)` |
| `sumOfDigits(1)` | 1 | `sumOfDigits(0)` |
| `sumOfDigits(0)` | - | 0 döner |

- Değerler yukarı doğru toplanır: $0 + 1 + 2 + 3 + 4 = 10$.
- Farklı durma koşulları da yazılabilir.

### Örnek 3: Palindrom

- Palindrom: tersten okunduğunda da aynı olan sayı ya da dizgi. 2002 ve 57675 palindromdur, 1234 değildir.
- Öğrencilere 3-4 dakika süre verildi, sonra çözüm anlatıldı.
- Yöntem: sayı özyinelemeli olarak ters çevrilir. `sum` değişkeni `static` tanımlanır, böylece her çağrıda sıfırlanmaz ve değerini korur.

```
int reverse(int number)
{
    static int sum = 0;
    int tmp;
    if (number != 0) {
        tmp = number % 10;
        sum = sum * 10 + tmp;
        return reverse(number / 10);
    }
    return sum;
}
```

| Çağrı | `tmp` | `sum` |
|---|---|---|
| `reverse(2002)` | 2 | 2 |
| `reverse(200)` | 0 | 20 |
| `reverse(20)` | 0 | 200 |
| `reverse(2)` | 2 | 2002 |
| `reverse(0)` | - | 2002 döner |

- Ana programda `reverse(x) == x` ise sayı palindromdur.

### Örnek 4: Özyinelemeli asallık testi

```
int isPrime(int n, int i)
{
    if (n <= 2)
        return (n == 2) ? 1 : 0;
    if (n % i == 0)
        return 0;
    if (i * i > n)
        return 1;
    return isPrime(n, i + 1);
}
```

- İlk çağrı `isPrime(n, 2)` olmalı. 3 ile başlanırsa 2'ye bölünme hiç kontrol edilmez.
- `return` kullanıldığı için `else if` yazmaya gerek yok.
- `isPrime(11, 2)`: $11 \% 2 \neq 0$, $4 \leq 11$, `isPrime(11, 3)` çağrılır. $11 \% 3 \neq 0$, $9 \leq 11$, `isPrime(11, 4)` çağrılır. $11 \% 4 \neq 0$, $16 > 11$, 1 döner. 11 asaldır.
- `isPrime(12, 2)`: ilk adımda $12 \% 2 == 0$, 0 döner. Fonksiyon bir kez çağrılır.
- Bu sürüm $\sqrt{n}$'e kadar bütün sayıları dener. Önce 2 kontrol edilip `i + 2` ile yalnız tek sayılara bakılarak iyileştirilebilir.

### Alıştırma: Kombinasyonlar

- $n$ elemanlı, elemanları tekrarsız bir dizi ve bir $r$ değeri veriliyor. Dizinin bütün $r$'li kombinasyonları özyinelemeli bir fonksiyonla yazdırılmalı.
- $\{1, 2, 3, 4, 5\}$ ve $r = 3$ için $\binom{5}{3} = 10$ kombinasyon vardır: 123, 124, 125, 134, 135, 145, 234, 235, 245, 345.
- Sıra önemli değil. 1, 2, 3 yazıldıysa 3, 2, 1 ayrıca yazılmaz.
- Dizi ve $r$ parametre olmalı. Örneğin $\{5, 7, 12, 28, 92, 89\}$ ve $r = 2$ ya da $r = 4$.

### Dizi ile bağlı listenin karşılaştırılması

- Dizi, aynı tipteki verileri bellekte arka arkaya (contiguous memory) saklar. Örneğin 1000 adresinde `a[0]`, 1004 adresinde `a[1]`.
- Dizinin avantajı: adresle doğrudan erişim.
- Dizinin dezavantajları:
  - Büyük veri için arka arkaya yeterli boş alan olmayabilir. Örneğin 999 bin elemanlık yer bulunur ama bir milyonluk bulunmaz.
  - Araya ekleme ve aradan silmede elemanları kaydırmak gerekir.
- Bağlı liste, belleğin dağınık boş alanlarını tek tek kullanabilir.
- Bağlı listenin dezavantajı: her düğüm, değerin yanında bir sonraki düğümün adresini de saklar. Son düğüme ulaşmak için bütün düğümler gezilir.
- Bazı uygulamalarda son düğümün adresi `tail` olarak ayrıca tutulur. Böylece sona ulaşmak için listeyi gezmek gerekmez.

### Bağlı listeyi gerçekleştirme yolları

- İki dizi ile: bir dizi değerleri, diğeri her elemandan sonra gelen elemanın dizideki indisini tutar. İki satırlı ya da iki sütunlu bir matrisle de yapılabilir. Az kullanılır, çünkü bağlı listeyi seçme nedeni zaten arka arkaya yer olmamasıdır.
- `struct` ile: en çok kullanılan ve verimli yöntem.

```c
struct node {
    int value;
    struct node *next;
};

struct node *head = NULL;

struct node *createNode(void)
{
    struct node *p;
    p = (struct node *)malloc(sizeof(struct node));
    if (p == NULL) {
        printf("Bellek ayrilamadi");
        exit(1);
    }
    return p;
}
```

- `value` alanı temsilidir. Bir öğrencinin bilgileri gibi bir başka `struct` da olabilir.
- Çift yönlü liste için `struct node *prev;` alanı eklenir.
- `malloc` `NULL` dönerse tek bir düğüm için bile yer kalmamıştır, kullanıcıya hata verilmelidir.

### Gezinme, arama ve güncelleme

```c
void printList(struct node *head)
{
    struct node *p;
    for (p = head; p != NULL; p = p->next)
        printf("%d ", p->value);
}
```

- Döngüye bir sayaç eklenirse listedeki düğüm sayısı bulunur.
- Bir öğrenci, fonksiyona verilen bir dizinin boyutunun `NULL` görene kadar sayılarak bulunup bulunamayacağını sordu. Hoca, bağlı listede `NULL`'a kadar düğümleri sayarak eleman sayısının bulunabileceğini söyledi.
- Bir öğrenci `p`'nin ne kadar ilerleyeceğini sordu. `p = p->next` ile `p` doğrudan sonraki düğümün adresine gider, `sizeof` ile hesap yapmaya gerek yoktur.
- Aynı döngüye bir `if` eklenerek arama, değer değiştirilerek güncelleme yapılır.

**Ekleme (sıralı liste)**

- Örnek liste: 3 → 5 → 9. Üç durum vardır.
- Başa ekleme (2 geliyor): yeni düğümün `next`'i eski `head`'i gösterir, `head` yeni düğümü gösterir.
- Araya ekleme (4 geliyor): 4'ten büyük ilk düğüm (5) bulunur. Önceki düğüm (3) yeni düğümü, yeni düğüm 5'i gösterir. Kaydırma yapılmaz, yalnız bağlantılar değişir.
- Sona ekleme (12 geliyor): son düğüme gelinir. Son düğümün `next`'i yeni düğümü gösterir, yeni düğümün `next`'i `NULL` kalır.

```
void insertSorted(int value)
{
    struct node *p, *prev, *current;
    p = createNode();
    p->value = value;
    p->next = NULL;
    if (head == NULL)
        head = p;
    else if (p->value < head->value) {
        p->next = head;
        head = p;
    }
    else {
        prev = head;
        current = head->next;
        while (current != NULL && current->value < p->value) {
            prev = current;
            current = current->next;
        }
        prev->next = p;
        p->next = current;
    }
}
```

- Bir diziden bağlı liste oluşturulurken de aynı fonksiyon her eleman için çağrılır.

**Silme**

- Örnek liste: 5 → 9 → 11.
- Önce liste boş mu kontrol edilir. Boşsa "liste boş" mesajı verilir.
- İlk düğüm siliniyorsa: `tmp = head`, `head = head->next`, `free(tmp)`.
- Değilse `prev` ve `current` ile ilerlenir. Düğüm bulunursa `prev->next = current->next` yapılır ve `current` serbest bırakılır.
- `current` `NULL` olduysa değer listede yoktur, "not found" yazılır.

```
void deleteNode(int value)
{
    struct node *tmp, *prev, *current;
    if (head == NULL) {
        printf("Liste bos");
        return;
    }
    if (head->value == value) {
```

```
        tmp = head;
        head = head->next;
        free(tmp);
        return;
    }
    prev = head;
    current = head->next;
    while (current != NULL && current->value != value) {
        prev = current;
        current = current->next;
    }
    if (current == NULL)
        printf("not found");
    else {
        prev->next = current->next;
        free(current);
    }
}
```

- Bu döngü sırasız liste içindir. Liste sıralıysa koşul `current->value < value` olur ve döngü sonrasında değer eşit mi diye ayrıca kontrol edilir.
- Arama kısmı ekleme ve silmede benzer olduğu için ayrı bir fonksiyon olarak yeniden kullanılabilir. Diğer kısımlar birbirinden farklıdır.

**Bağlı liste türleri**

- Tek yönlü (singly linked list): yalnız ileri gidilir. Son düğüm `NULL`'u gösterir.
- Çift yönlü (doubly linked list): her düğümde `prev` ve `next` vardır. İlk düğümün `prev`'i ve son düğümün `next`'i `NULL`'dur. İleri ve geri hareket edilebilir.
- Çift yönlü listede silme: silinen düğümün önceki düğümünün `next`'i ve sonraki düğümünün `prev`'i güncellenir.
- Dairesel (circular linked list): son düğümün `next`'i ilk düğümü gösterir. İşletim sistemlerinde süreç zamanlamada (process scheduling) kullanılır: süresi biten süreçten sonrakine geçilir, liste bitince başa dönülür.
- Dairesel çift yönlü: ek olarak ilk düğümün `prev`'i son düğümü gösterir.
- Dairesel listede gezinme `current->next == head` olunca durdurulur.
- Ek adres maliyeti: değer büyükse (40 byte gibi) iki adres eklemek boyutu üç katına çıkarmaz. Örneğin 40 byte'lık veriye $2 \times 4$ byte adres eklenince 48 byte olur.

**Kullanım alanları**

- Tarayıcıda sayfalar arasında ileri-geri gezinme.
- Telefonda fotoğraflar arasında gezinme.
- Müzik uygulamasında sonraki ya da önceki şarkıya geçme.
- İşletim sisteminde Alt+Tab ile uygulamalar arasında geçiş.
- İleride ağaç gezinmeleri (inorder, preorder) ve graf gezinmeleri (DFS, BFS) bağlı listelerle ilişkilendirilecek.
- Sonraki ders: bağlı listenin avantaj ve dezavantajlarının özeti, ardından yığın (stack) ve kuyruk (queue).

**Ders sonrası notlar**

- Silme akış diyagramında aradaki düğümü bağlantıdan çıkardıktan sonra `free(current)` unutulmuştu. Bir öğrenci fark etti, hoca ekledi.
- Bir öğrenci, akış diyagramındaki "head value" ifadesini sordu. `head` yalnız adres tutar. `head->value`, `head`'in gösterdiği düğümün değeridir.
- Palindrom çözümündeki `static int sum` her çağrıda sıfırlanmaz, ama program boyunca da sıfırlanmaz. Fonksiyon ikinci kez başka bir sayı için çağrılırsa `sum` önceki sonuçtan devam eder ve sonuç yanlış çıkar. Fonksiyonu birden çok kez kullanmak için `sum`'ı ikinci

bir parametre olarak geçirmek daha güvenlidir: `reverse(number / 10, sum * 10 + number % 10)`.

- Hoca `struct` tanımında `typedef` kullanılamayacağını söyledi. Doğrusu: `typedef` yapılabilir, ama `struct`'ın içinde kendine işaret eden alan için `struct node *` yazılmalıdır. Çünkü `typedef` adı o noktada henüz tanımlı değildir.
- Hoca {1, 2, 3, 4, 5} ve $r = 3$ örneğinde tahtaya yalnız 6 kombinasyon yazdı: 123, 124, 125, 234, 235, 345. Doğrusu $\binom{5}{3} = 10$ kombinasyondur. 134, 135, 145 ve 245 eksikti.
- Hoca, 4 ile başlayan kombinasyonların ayrıca yazılıp yazılmayacağı sorusunda önce emin olamadı, sonra kontrol etti. Kombinasyonda sıra önemsizdir; 4, 1, 2 ile 1, 2, 4 aynı kombinasyondur.
- Dairesel listede arama yaparken aranan değer yoksa döngü sonsuza kadar sürebilir. Bu yüzden döngü, başa dönülünce (`current == head`) durdurulmalıdır.
- Hoca sıralı eklemenin akış diyagramında, liste boşken `head = p` atamasından sonra bir de `head->value = value` yazdı ve bunu adres ile değer aktarımını ayrı göstermek için yaptığını söyledi. `p->value` önceden atandığı ve `head` artık `p`'yi gösterdiği için bu satır gereksizdir. Yukarıdaki kodda yoktur.
- Asallık testinin düzeltilmiş sürümünde (döngü `i < sinir`, ardından `x % i != 0` ise asal) 5 ve üstü sayılar için sonuç doğrudur (python ile 2000'e kadar denendi). Ama 3 için `i = 3` olduğundan `3 % 3 == 0` çıkar ve 3 asal değil sanılır. 2 de çift olduğu için asal değil sayılır. 1, 2 ve 3 ayrıca ele alınmalıdır.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.